

U.E 5.1 - Intelligence Artificielle : Machine learning

Promotion FISE 2027

ENSTA BRETAGNE
2 Rue François Verny
29200 Brest Cedex 9, France



BARROS ALVES Luiz Felipe
luiz.barros@ensta.fr

Sommaire

1	Linear regression with one variable	2
1.1	Plotting the Data	2
2	Gradient Descent	2
2.1	Computing the cost $J(\theta)$	2
2.2	Gradient descent	3
2.3	Visualizing $J(\theta)$	5
3	Linear regression with multiple variables	6
3.1	Feature Normalization	6
3.2	Learning with gradient descent	7
3.2.1	Compute cost and gradients	7
3.2.2	Selecting learning rates	9
3.2.3	Predict the price for a new house	9
3.3	Optional exercise: Normal Equations	10
3.4	Some questions, you have to answer...	11

1 Linear regression with one variable

1.1 Plotting the Data

- How many features are involved in this problem?

We have one feature (population of a city) and one target variable (profit of a food truck).

- How much data is involved in this problem?

We have 97 data points (97 cities) in the dataset.

- What is the problem from a machine learning point-of-view?

Supervised/Unsupervised?

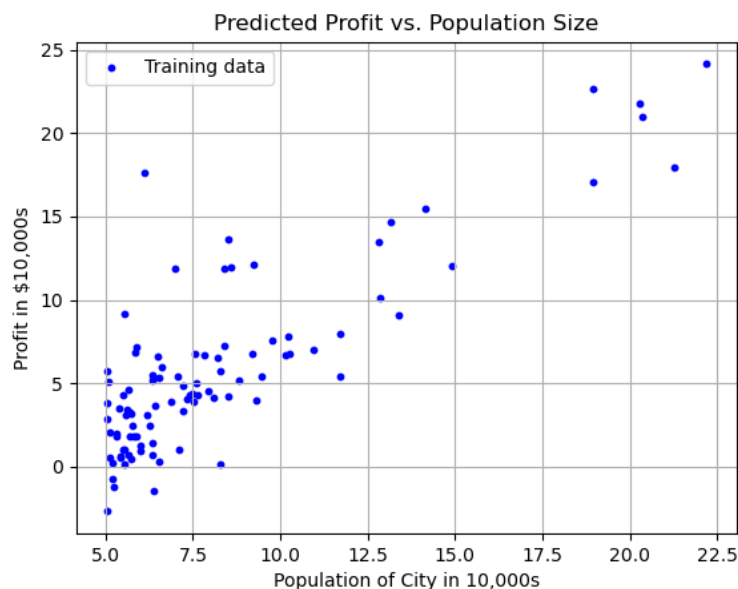
Supervised learning, as we have input-output pairs (population-profit).

Regression/Classification?

Regression, as we are predicting a continuous value (profit) based on the input feature (population).

Linear/?on-linear problem

Linear problem, as we are trying to fit a linear model to the data.



2 Gradient Descent

2.1 Computing the cost $J(\theta)$

```

1 import numpy as np
2
3 def computeCost(X, y, theta):
4     """
5         computes the cost of using theta as the parameter for
6         linear
7         regression to fit the data points in X and y
8     """
9     m = y.size
10    J = 0.
11
12    # =====YOUR CODE HERE =====
13    # Instructions: Compute the cost of a particular choice of
14    # theta
15
16    #h = float(theta[0,0]) + float(theta[1,0])*X[:,1]
17    #h = h.reshape((m,1))
18    h = X @ theta
19    J = (1/(2*m)) * np.sum((h-y)**2)
20
21    # =====
22
23    return J

```

2.2 Gradient descent

```

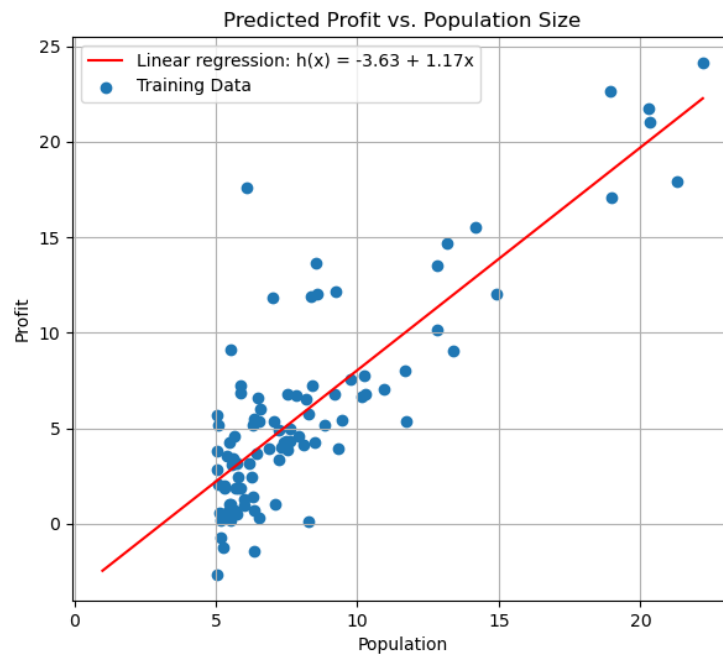
1 import numpy as np
2 from computeCost import computeCost
3
4 def gradientDescent(X, y, theta, alpha, num_iters):
5     """
6         Performs gradient descent to learn theta
7         theta, cost_history, theta_history = gradientDescent(X,
8         y, theta, alpha, num_iters) updates theta by
9         taking num_iters gradient steps with learning rate
10        alpha
11    """
12    # Initialize some useful values
13    m = y.size # number of training examples
14    n = theta.size # number of parameters
15    cost_history = np.zeros(num_iters) # cost over iters
16    theta_history = np.zeros((n,num_iters)) # theta over iters
17
18    for i in range(num_iters):
19        # ===== YOUR CODE HERE =====
20
21

```

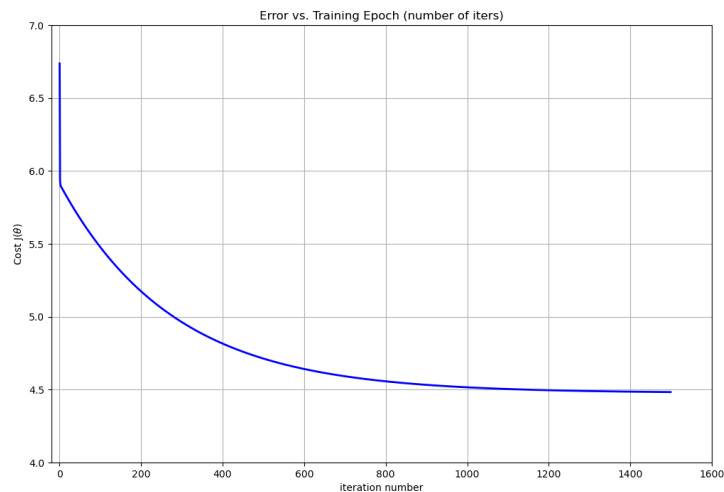
```
18 # Instructions: Perform a single gradient step on the
    parameter vector
19 #             theta.
20 #
21 # Hint: While debugging, it can be useful to print out the
    values
22 #             of the cost function (computeCost) and gradient
    here.
23
24 h = X @ theta
25
26 theta[0,0] = theta[0,0] - (alpha/m)*np.sum( ((h-y)*X
   [:,0].reshape((m,1))) )
27 theta[1,0] = theta[1,0] - (alpha/m)*np.sum(((h-y)*X
   [:,1].reshape((m,1))))
28
29 cost_history[i] = computeCost(X, y, theta)
30 theta_history[:,i] = theta.reshape((2,))
31
32 # =====
33
34 return theta, cost_history, theta_history
```

In this part, we can see the datas (X and y) are constant. However, the parameters (theta) are updated at each iteration of the gradient descent algorithm. The cost function $J(\theta)$ is computed at each iteration to monitor the convergence of the algorithm. The goal is at each iteration to update the parameters (theta) in the direction that reduces the cost function more quickly possible to the global minimum.

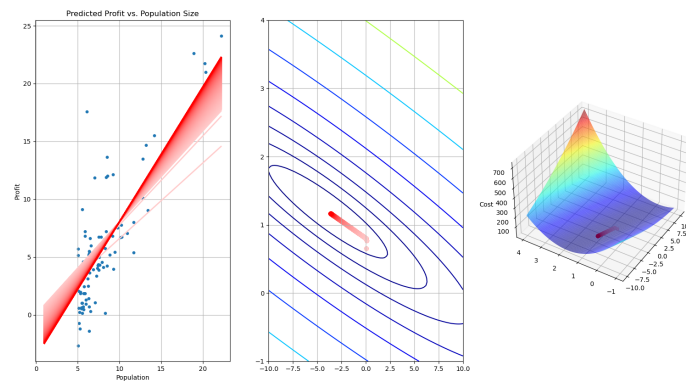
Training data with linear regression fit :



2.3 Visualizing $J(\theta)$



In this part, we can see the cost function $J(\theta)$ is a convex function, which means it has a single global minimum. The contour plot shows the values of $J(\theta)$ for different values of θ_0 and θ_1 . The red dot represents the optimal parameters (theta) that minimize the cost function.



3 Linear regression with multiple variables

3.1 Feature Normalization

```

1 import numpy as np
2
3
4 def featureNormalize(X):
5     """
6         returns a normalized version of X where
7         the mean value of each feature is 0 and the standard
8         deviation
9         is 1. This is often a good preprocessing step to do
10        when
11        working with learning algorithms.
12    """
13    X_norm, mu, sigma = 0,0,0
14    # ===== YOUR CODE HERE =====
15
16    # Instructions: First, for each feature dimension, compute
17    # the mean
18    # of the feature and subtract it from the
19    # dataset,
20    # storing the mean value in mu. Next,
21    # compute the
22    # standard deviation of each feature and
23    # divide
24    # each feature by it's standard deviation,
25    # storing
26    # the standard deviation in sigma.
27    #
28    # Note that X is a matrix where each column
29    # is a
30    # feature and each row is an example. You
31    # need

```

```

22         # to perform the normalization separately
23         for
24         # each feature.
25         # Hint: You might find the 'mean' and 'std' functions
26         # useful.
27
28
29         mu = np.mean(X, axis=0)
30         sigma = np.std(X, axis=0)
31         X_norm = (X - mu) / sigma
32
33
34
35         # =====
36
37         return X_norm, mu, sigma

```

3.2 Learning with gradient descent

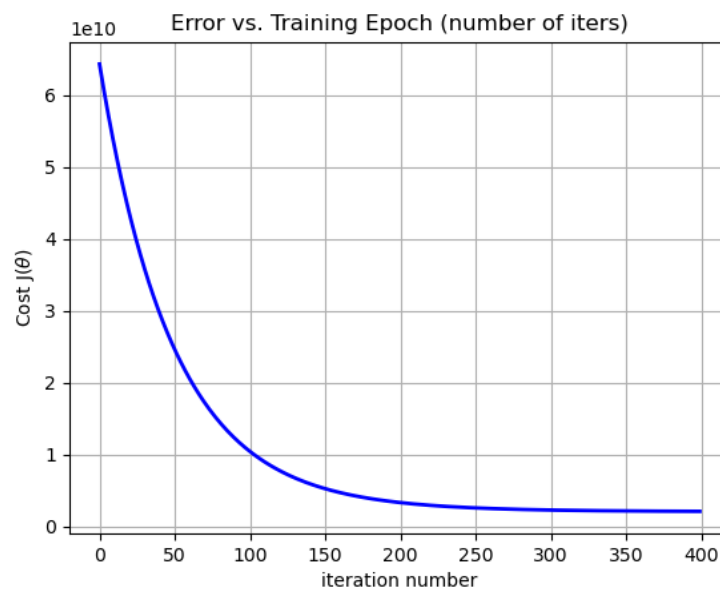
3.2.1 Compute cost and gradients

```

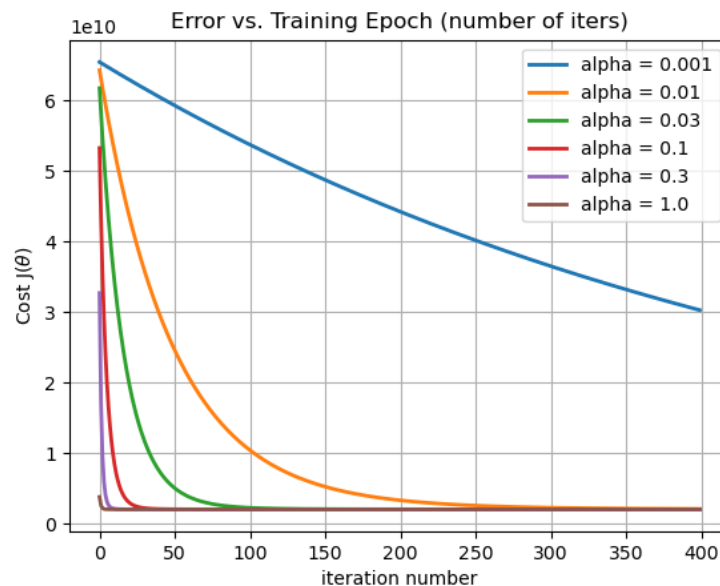
1 import numpy as np
2 from computeCostMulti import computeCostMulti
3
4 def gradientDescentMulti(X, y, theta, alpha, num_iters):
5     """
6     Performs gradient descent to learn theta
7     theta = gradientDescent(x, y, theta, alpha, num_iters)
8     updates theta by
9     taking num_iters gradient steps with learning rate
10    alpha
11    """
12    # Initialize some useful values
13    m = y.size # number of training examples
14    n = theta.size # number of parameters
15    cost_history = np.zeros(num_iters)
16    theta_history = np.zeros((n,num_iters))
17
18    for i in range(num_iters):
19        # ===== YOUR CODE HERE =====
20
21        # Instructions: Perform a single gradient step on the
22        # parameter vector
23        # theta.
24
25        # Hint: While debugging, it can be useful to print out the
26        # values

```

```
22     #         of the cost function (computeCost) and gradient
23     here.
24     #
25     h = X @ theta
26     grad = (1/m) * (X.T @ (h - y))
27     theta = theta - alpha * grad
28
29     cost_history[i] = computeCostMulti(X, y, theta)
30     theta_history[:,i] = theta.reshape((n,))
31
32     # =====
33
34     return theta, cost_history, theta_history
```



3.2.2 Selecting learning rates



The range of values for the learning rate α was between 0.001 and 1.0 if the we passed of 1.0 the cost function $J(\theta)$ diverged and the algorithm did not converge to the global minimum. The best learning rate was 1.0, as it converged faster to the global minimum than the other learning rates.

3.2.3 Predict the price for a new house

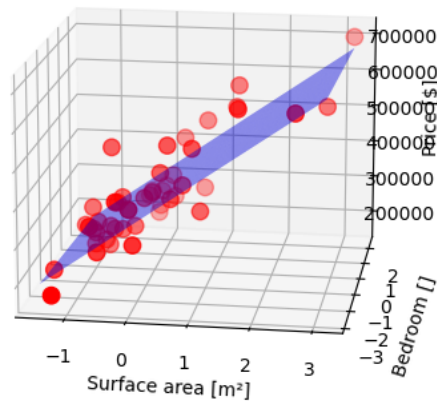
```

1 import numpy as np
2
3 # Estimate the price of a 1650 sq-ft, 3 br house
4 def predict_multi(house_features, mu, sigma, theta):
5
6     # ===== YOUR CODE HERE
7     # =====
8     # You should use the learned model with its
9     # characteristics (mu, sigma, theta)
10    # to predict the price of a house based on its features (
11    # surface and bedrooms)
12
13    house_features_norm = (house_features - mu) / sigma
14    house_features_norm = np.concatenate((np.ones((1, 1)),
15        house_features_norm), axis=1)
16    price = house_features_norm.dot(theta)
17    print(f'shapes {np.shape(theta)} {np.shape(
18        house_features_norm)}')
19    return price

```

- Predicted price of a 1650 sq-ft, 3 br house (using gradient descent): 293081.4643349

Plot to visualize the plan created by the function theta



3.3 Optional exercise: Normal Equations

```

1 import numpy as np
2
3
4 def normalEqn(X,y):
5     """ Computes the closed-form solution to linear regression
6         normalEqn(X,y) computes the closed-form solution to
7         linear
8         regression using the normal equations.
9     """
10
11     # ===== YOUR CODE HERE
12     # =====
13     # Instructions: Complete the code to compute the closed
14     # form solution
15     # to linear regression and put the result in
16     # theta.
17     #
18
19     t = np.linalg.pinv(X.T @ X) @ X.T @ y
20
21     # =====
22
23     return t

```

Solving with normal equations...

Theta computed from the normal equations: $\begin{bmatrix} 89597.90954355 \\ 139.21067402 \\ -8738.01911255 \end{bmatrix}$

- Predicted price of a 1650 sq-ft, 3 br house (using normal equations): 293081.464335

We can see that the predicted price of a 1650 sq-ft, 3 br house is very close using gradient descent and normal equations. The normal equations method is more efficient for small datasets, while gradient descent is more efficient for large datasets.

3.4 Some questions, you have to answer...

- Définissez les termes :

Approches supervisées/non-supervisées

Apprentissage supervisé : l'algorithme apprend à partir d'exemples étiquetés (input-output pairs) pour prédire la sortie pour de nouvelles entrées.
Apprentissage non-supervisé : l'algorithme apprend à partir de données non étiquetées pour trouver des structures ou des motifs dans les données.

Régression/ Classification

Régression : On utilise ça quand on veut deviner un chiffre précis (comme le prix d'une maison ou une température). Classification : On utilise ça quand on veut deviner une catégorie (par exemple, dire si une image montre un chat ou un chien).

Descripteurs/features

Ce sont les informations qu'on utilise pour faire notre choix. Pour notre exercice, c'est la taille de la maison et le nombre de chambres.

Cibles

C'est le résultat qu'on veut deviner. Ici, c'est le prix de la maison.

Modèle de régression linéaire

C'est une idée mathématique qui dit que les informations et le résultat sont liés par une ligne droite. Si la maison est plus grande, le prix monte de façon régulière.

- Représenter en un schéma général, les processus d'apprentissage et de prédiction.

L'apprentissage (créer le modèle) :

Données connues (Descripteurs + Cibles)

→ L'ordinateur s'entraîne et corrige ses erreurs

→ On obtient un modèle prêt.

La prédiction (utiliser le modèle) :

Nouvelles données (Descripteurs d'une nouvelle maison, sans le prix)

→ Le modèle fait son calcul

→ Il nous donne le prix deviné.

- Comment fonctionne l'apprentissage? Par quels moyens? A quoi sert la fonction de coût? Comment est-résolu le problème? Connaissez vous d'autres moyens de le résoudre?

Comment ça marche ? Au début, le modèle devine le prix un peu au hasard. Ensuite, il regarde s'il a fait une grosse erreur ou pas. Il change un peu ses réglages pour être meilleur la prochaine fois. Il fait ça des centaines de fois.

À quoi sert la fonction de coût ? C'est comme une note qui mesure les erreurs du modèle. Le but de l'algorithme est d'avoir l'erreur la plus petite possible.

Comment le problème est résolu ? On a utilisé la méthode de la descente de gradient. C'est une technique où l'ordinateur avance petit à petit pour descendre vers l'erreur la plus petite.

D'autres moyens de le résoudre ? Oui, on a aussi programmé l'équation normale. C'est un grand calcul mathématique qui trouve la meilleure réponse d'un seul coup, sans avancer pas à pas.

- Pourquoi faut-il parfois normaliser les descripteurs (features)?

On fait ça quand on a des données avec des chiffres très différents. Par exemple, la taille de la maison est très grande (1650) et le nombre de chambres est très petit (3). L'algorithme a du mal à comparer ces choses et ça ralentit beaucoup les calculs. Normaliser, c'est transformer ces chiffres pour les mettre sur la même échelle (par exemple, tout mettre entre -1 et 1). Grâce à ça, le programme trouve la solution beaucoup plus vite.